TATA TECHNOLOGIES LTD. • TECH PULSE FY-26
AI/ML Capstone Project

PROJECT SYNOPSIS

Secure Code Debugging and Review Assistant

Student Details

Field	Details
Institute / University	PCCOE (confirm official institute name)
Programme / Batch	Applied AI/ML Capstone — TechPulse FY-26
Student Name	Janvee Ghadge
PRN	123B1D060
Case Study ID (CS1–CS5)	Case Study 4 (confirm official ID)
Project Title / Topic	Secure Code Debugging and Review Assistant

1. Case Study ID and Project Title

Item	Details
Case Study ID	Case Study 4 (confirm the official CS1–CS5 mapping with faculty)
Project title	Secure Code Debugging and Review Assistant

2. Introduction and Problem Statement

Manual code review can be time-consuming, and developers may overlook security weaknesses or code-quality issues. Static-analysis tools identify suspicious patterns but do not always give enough context for a developer to understand and fix a finding. This project combines Python static analysis with retrieval of secure-coding guidelines and a downloadable report to support preliminary code review.

Intended users: students, Python developers, reviewers and educators who need an accessible first-pass review tool. The application is advisory and does not replace manual review, testing or a professional security audit.

3. Objectives

- **O1.** Develop a Streamlit interface for submitting Python source code for review.
- **O2.** Use Bandit and Ruff to identify potential security and code-quality issues.
- **O3.** Retrieve relevant secure-coding guidance from a local knowledge base using Sentence Transformers and ChromaDB.

O4. Present findings with severity, line numbers, evidence and relevant guideline passages.

O5. Generate a downloadable Markdown report and evaluate the prototype using vulnerable, clean and malformed code samples.

4. Scope

Table 1. Scope of work

In scope — to be implemented	Out of scope — will not be implemented
• Python source-code review using available static-analysis tools. • Local indexing and retrieval of approved or synthetic secure-coding guidelines. • Display of findings and guideline evidence through Streamlit. • Markdown report export and evaluation using a small test-case set.	• Guaranteed detection of all vulnerabilities or replacement of expert security audits. • Automatic execution of submitted code or automatic application of fixes. • Review of confidential/proprietary code through an external API without authorization. • Claims of measured precision/recall until a larger labelled ground-truth dataset is evaluated.

5. Proposed Approach

5.1 Technology Choices

Table 2. Proposed / current choices

Component	Proposed / current choice
Pre-trained model	Optional OpenAI model via API; code default is gpt-4.1-mini. Not configured in the current tested run.
Embedding model	sentence-transformers/all-MiniLM-L6-v2.
Vector store	ChromaDB PersistentClient; local chroma_db directory; collection coding_guidelines.
RAG pipeline / AI-agent design	Finding-aware retrieval queries search local secure-coding guideline chunks. This is a RAG workflow, not a multi-agent system.
Service layer and UI	Python and Streamlit; main UI in app.py.
Tools / deterministic rules	Bandit and Ruff for static analysis; section-aware chunking; ChromaDB retrieval; Markdown report generation.

5.2 Proposed System Architecture

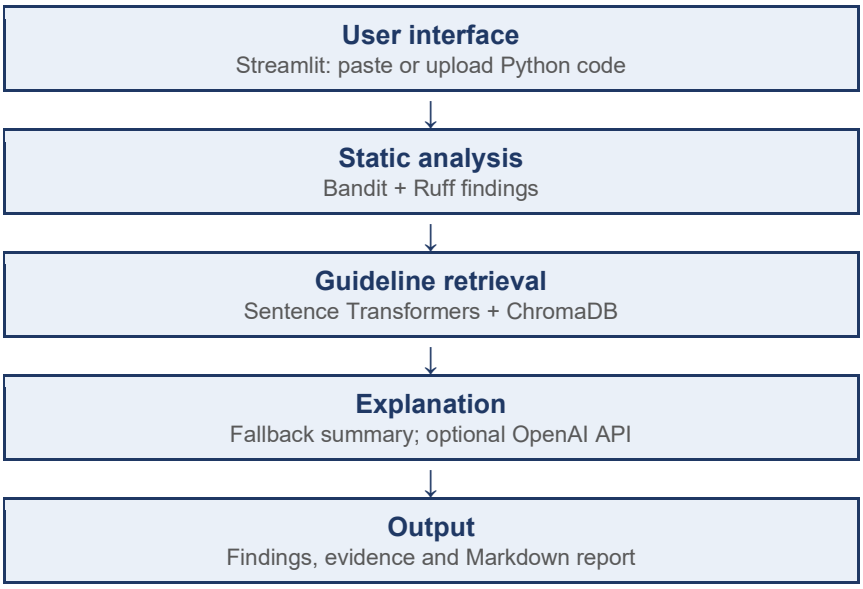


Figure 1. Proposed workflow of the Secure Code Review Assistant.

6. Input Data / Knowledge Base

Table 3. Knowledge base summary

Item	Description
Data / knowledge sources	Local Markdown/TXT guideline files under data/guidelines/; current demo file coding_guidelines_demo.md. Approved or synthetic examples only.
Approximate size and format	Small local Markdown document; the observed index contained 11 guideline chunks after section-aware indexing.
Preprocessing and EDA planned	Read text as UTF-8; separate sections using Markdown headings; split long sections into chunks of up to 180 words; retain source/chunk metadata; re-index after guideline changes.

7. Expected Outcomes

- Findings for suspicious code patterns, including severity, rule ID, line number and evidence.
- Relevant secure-coding guideline passages with source and chunk labels.
- Downloadable Markdown report and summary of limitations.
- A documented evaluation set with vulnerable, clean, syntax-error, safer-subprocess and input-validation scenarios.

8. Evaluation Plan

8.1 Method

Use the six recorded cases in evaluation/test_cases.csv. Compare actual analyzer output with expected behaviour, and retain screenshots and exported reports as evidence.

8.2 Current Status and Caveats

- The current CSV marks 6/6 cases as Pass; this is a test-case pass rate, not measured detection accuracy.
- TC01 and TC02 overlap, so they are not independent inputs.
- Retrieval relevance is inspected qualitatively.
- Formal precision, recall, latency and retrieval hit rate remain to be measured.

9. Responsible AI Measures

Table 4. Responsible AI measures

Area	Measure
Grounding and transparency	Retrieved evidence is shown with source/chunk information.
Privacy	Guideline retrieval and the vector index are local. The optional OpenAI API may transmit submitted code to an external provider; confidential code must not be submitted without authorization.
Human oversight	Findings and explanations are advisory and require human review and testing.

10. MLOps / Deployment Elements

The following are planned; they are not claimed as completed.

- **Planned:** Version source code and evaluation data with Git.
- **Planned:** Maintain requirements.txt and documented setup steps.
- **Optional future improvements:** Docker packaging and automated tests/CI.

11. Planned AI-Tool Usage

Table 5. AI tools and models

AI tool / model	Purpose	Artifact affected
ChatGPT / OpenAI assistant	Assistance with debugging, explanations, drafting and report structure; student must verify all generated material.	Code changes and project documentation; disclose accurately in final declaration.
Sentence Transformers: all-MiniLM-L6-v2	Create embeddings for guideline chunks and retrieval queries.	Local RAG pipeline; tested during retrieval.
OpenAI API model (optional)	Generate natural-language review explanations when API configuration is enabled.	Optional explanation function; not configured in current test run.

12. Student Declaration

- I will not change the approved case-study scope, major methodology or implementation approach without prior faculty approval.

- I confirm that the work submitted under this synopsis represents my own understanding and contribution, and I will be responsible for explaining every significant part of the selected case study and its implementation.

Name: Janvee Ghadge Date: _____